

WT EXPERIMENT - 04

Name	Class	Roll no
Nitish Bhosle	D15A	04
Aryan Dangat	D15A	12

Aim: Component Description and Arduino Working

Introduction

This document provides a detailed overview of the Vehicle Collision Detection System with GPS. The system utilizes motion sensors to detect vehicle collisions and provides real-time GPS location data to assist emergency responders. This project focuses on the hardware components used and the Arduino code implementation, excluding interfacing details.

Components Used

Arduino Uno

The Arduino Uno serves as the central processing unit. It reads data from the MPU6050 accelerometer, processes collision detection logic, retrieves GPS coordinates, and updates the LCD screen.

MPU6050 Accelerometer/Gyroscope

The MPU6050 sensor detects sudden acceleration changes, which indicate a potential collision. The sensor measures acceleration in three axes (X, Y, Z) and calculates the total acceleration.

NEO-6M GPS Module

The GPS module provides latitude and longitude coordinates of the vehicle. Upon collision detection, the system records the last valid GPS coordinates.

I2C LCD (16x2 Display)

A 16x2 I2C LCD is used to display status messages, including collision alerts and GPS coordinates. It communicates with the Arduino via the I2C protocol.

Breadboard and Jumper Wires

These components are used for prototyping and interconnecting the sensors and modules with the Arduino.

Arduino Uno Overview

The **Arduino Uno** is an open-source microcontroller board based on the **ATmega328P**. It is widely used in embedded systems and automation projects.

Key Features:

- **Microcontroller:** ATmega328P
- **Operating Voltage:** 5V
- **Digital I/O Pins:** 14 (6 PWM outputs)
- **Analog Input Pins:** 6
- **Flash Memory:** 32 KB
- **Clock Speed:** 16 MHz
- **Communication Protocols:** Supports SPI, I2C, and UART

Working Principle

Collision Detection Using the MPU6050 Sensor

The **MPU6050 accelerometer** continuously measures acceleration in the X, Y, and Z directions. The magnitude of the acceleration is calculated using the formula:

$$accel = \sqrt{x^2 + y^2 + z^2} \times 9.81$$

A **collision** is detected when the acceleration exceeds a predefined **threshold value** (e.g., **2.0g**).

Retrieving GPS Data from the NEO-6M Module

Once a collision is detected, the system **retrieves the last valid latitude and longitude coordinates** from the **GPS module**. These coordinates are used to pinpoint the vehicle's location.

Displaying Status Messages on the LCD

The **LCD alternates between displaying the message "COLLISION!"** and the **GPS latitude/longitude coordinates** every **2 seconds** to provide real-time situational awareness

Arduino Code Explanation

The Arduino code is structured into **collision detection**, **GPS data acquisition**, and **LCD display updates**.

```
cpp
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <MPU6050.h>
#include <SoftwareSerial.h>

#define COLLISION_THRESHOLD 2.0

MPU6050 mpu;
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2);  
SoftwareSerial gpsSerial(4, 3); // RX, TX for GPS module
```

```
float ax, ay, az;  
String latitude = "N/A", longitude = "N/A";
```

```
void setup() {  
  Serial.begin(9600);  
  gpsSerial.begin(9600);  
  
  Wire.begin();  
  lcd.init();  
  lcd.backlight();  
  lcd.setCursor(0, 0);  
  lcd.print("System Ready");  
  
  mpu.initialize();  
}
```

```
void loop() {  
  // Read MPU6050 acceleration values  
  mpu.getAcceleration(&ax, &ay, &az);  
  float accel = sqrt(ax * ax + ay * ay + az * az) / 16384.0; // Convert to g-force  
  
  // Collision detection  
  if (accel > COLLISION_THRESHOLD) {  
    Serial.println("Collision Detected!");  
    lcd.clear();  
    lcd.setCursor(0, 0);  
    lcd.print("COLLISION!");  
  
    // Fetch GPS Data  
    if (gpsSerial.available()) {  
      String gpsData = gpsSerial.readStringUntil('\n');  
      if (gpsData.indexOf("GPGGA") > 0) {  
        int latIndex = gpsData.indexOf(",") + 1;  
        latitude = gpsData.substring(latIndex, latIndex + 10);  
        longitude = gpsData.substring(latIndex + 12, latIndex + 22);  
      }  
    }  
  
    lcd.setCursor(0, 1);  
    lcd.print("Lat: " + latitude);  
    delay(2000);  
    lcd.clear();  
    lcd.setCursor(0, 0);  
    lcd.print("Long: " + longitude);  
    delay(2000);  
  }  
}
```

Code Breakdown

Libraries and Definitions

The following libraries are included:

- `<Wire.h>` for **I2C communication** (LCD and MPU6050).
- `<LiquidCrystal_I2C.h>` for controlling the **LCD screen**.
- `<MPU6050.h>` for interfacing with the **accelerometer/gyroscope**.
- `<SoftwareSerial.h>` for communicating with the **GPS module**.

The **collision threshold** is set to **2.0g**.

Global Object Initialization

- **MPU6050 mpu** → Initializes the accelerometer.
- **LiquidCrystal_I2C lcd(0x27, 16, 2)** → Configures the LCD at address **0x27**.
- **SoftwareSerial gpsSerial(4, 3)** → Defines GPS communication using **pins 4 and 3**.

Setup Function

- Initializes **serial communication** for debugging.
- Starts **I2C communication** for the LCD.
- Displays **"System Ready"** on the LCD at startup.
- Initializes the **MPU6050 sensor**.

Loop Function

1. Reads acceleration values from the **MPU6050**
2. Calculates the acceleration magnitude
3. Detects a collision when acceleration exceeds the threshold
4. Retrieves the last known GPS coordinates
5. Displays the collision alert and GPS location on the LCD

Challenges & Solutions

MPU6050 I2C Communication Issues

- **Problem:** The **MPU6050 sensor** was sometimes **unresponsive** over I2C.
- **Solution:** The sensor's **I2C address (0x68/0x69)** was verified using an **I2C scanner**.

GPS Signal Delay

- **Problem:** The GPS module takes time to acquire valid **latitude and longitude** data.
- **Solution:** Implemented **continuous background GPS parsing** to store the **last known location**.

Limited LCD Screen Space

- **Problem:** The **LCD can only display two lines at a time**, making it difficult to

show all details.

- **Solution:** The system **alternates between latitude and longitude every 2 seconds**.

False Collision Triggers

- **Problem:** The MPU6050 sometimes detected **false collisions** due to vibrations.
- **Solution:** The **collision threshold was fine-tuned between 1.5g–3.0g** through testing.

Conclusion

This document outlines the implementation of a Vehicle Collision Detection System with GPS. The system efficiently detects collisions using the MPU6050 accelerometer and logs real-time GPS coordinates via the NEO-6M GPS module. The I2C LCD display ensures clear status updates.

This system is **vital for automotive safety, fleet management, and emergency response applications**.